

Working with Real Data

Here we are going to see:

- Manipulating data streams
- Working with flat files
- Working with unstructured files
- Interacting with relational databases
- Using NoSql as a data source
- Interacting with web-based data

Data science applications require data by definition. However, data is messy. It appears in all sorts of places, in many different forms, and you can interpret it in many different ways. Every organization has a different method of viewing data and stores it in a different manner as well. In short, before you can do any data science work, you must discover how to access the data in all its myriad forms. Real data requires a lot of work to use and fortunately, Python is up to the task of manipulating it as needed

Understand the techniques required to access data in a number of forms and locations. For example, memory streams represent a form of data storage that your computer supports natively; flat files exist on your hard drive; relational databases commonly appear on networks and web-based data usually appears on the Internet.

Now we are going to see how to access data in the formats you most commonly encounter when working with real-world data

Uploading, Streaming, and Sampling Data

- Storing data in local computer memory represents the fastest and most reliable means to access it. The data could reside anywhere. However, you don't actually interact with the data in its storage location. You load the data into memory from the storage location and then interact with it in memory.

Uploading small amounts of data into memory

- The most convenient method that you can use to work with data is to load it directly into memory.
- When you load a file (of any type), the entire dataset is available at all times and the loading process is quite short

In [71]:

```
%pwd
```

Out[71]:

```
'C:\\Users\\KHYATI'
```

In [72]:

```
with open("Colors.txt", 'r') as open_file:
    print('Colors.txt content:', open_file.read())
```

```
Colors.txt content: color value
Red 1
Orange 2
Yellow 3
Green 4
Blue 5
Purple 6
Black 7
White 8
```

Note:

- The **with statement** itself ensures proper acquisition and release of resources.
- In above example it begins by using the `open()` method to obtain a file object. The `open()` function accepts the filename and an access mode.
- It then uses the `read()` method of the file object to read all the data in the file. If you were to specify a size argument as part of `read()`, such as `read(15)`, Python would read only the number of characters that you specify or stop when it reaches the End Of File (EOF).

Streaming large amounts of data into memory

- Some datasets will be so large that you won't be able to fit them entirely in memory at one time. In addition, you may find that some datasets load slowly because they reside on a remote site.
- Streaming answers both needs by making it possible to work with the data a little at a time.

In [73]:

```
with open("Colors.txt", 'r') as open_file:
    for observation in open_file:
        print('Reading Data: ', observation)
```

```
Reading Data:  color value

Reading Data:  Red 1

Reading Data:  Orange 2

Reading Data:  Yellow 3

Reading Data:  Green 4

Reading Data:  Blue 5

Reading Data:  Purple 6

Reading Data:  Black 7

Reading Data:  White 8
```

Note:

As the code performs data reads in the for loop, the file pointer moves to the next record. Each record appears one at a time in observation. The code outputs the value in observation using a print statement.

Sampling data

- Data streaming obtains all the records from a data source. You may find that you don't need all the records. You can save time and resources by simply sampling the data.
- This means retrieving records a set number of records apart, such as every fifth record, or by making random samples.

In [74]:

```
n = 2
with open("Colors.txt", 'r') as open_file:
    for j, observation in enumerate(open_file):
        if j % n == 0:
            print('Reading Line: ' + str(j) + ' Content: ' + str(observation))
```

Reading Line: 0 Content: color value

Reading Line: 2 Content: Orange 2

Reading Line: 4 Content: Green 4

Reading Line: 6 Content: Purple 6

Reading Line: 8 Content: White 8

Note:

in this case, the application uses `enumerate()` to retrieve a row number. When `j % n == 0`, the row is one that you want to keep and the application outputs the information.

Random Samples

-The `random()` method outputs a value between 0 and 1. However, Python randomizes the output so that you don't know what value you receive. The `sample_size` variable contains a number between 0 and 1 to determine the sample size. For example, 0.25 selects 25 percent of the items in the file.

In [94]:

```
from random import random
sample_size = 0.25
with open("Colors.txt", 'r') as open_file:
    for j, observation in enumerate(open_file):
        if random() <= sample_size:
            print('Reading Line: ' + str(j) + ' Content: ' + str(observation))
```

Reading Line: 0 Content: color value

Reading Line: 1 Content: Red 1

Reading Line: 4 Content: Green 4

Accessing Data in Structured Flat-File Form

- Real-world data usually appears in a file of some type. A flat file presents the easiest kind of file to work with.
- A problem with using native Python techniques is that the input isn't intelligent. For example, when a file contains a header, Python simply reads it as yet more data to process, rather than as a header. You can't easily select a particular column of data.
- pandas library interpret (parse) the flat-file data to make it easier to manipulate.

Reading from a text file

The pandas I/O API is a set of top level reader functions accessed like `pandas.read_csv()` that generally return a pandas object. The corresponding writer functions are object methods that are accessed like `DataFrame.to_csv()`.

- to read text file we have two functions `read_table()` and `read_csv()`.

In [76]:

```
import pandas as pd
color_table = pd.io.parsers.read_table("Colors.txt")
print(color_table)
```

```
color value
0      Red 1
1    Orange 2
2    Yellow 3
3     Green 4
4      Blue 5
5    Purple 6
6     Black 7
7     White 8
```

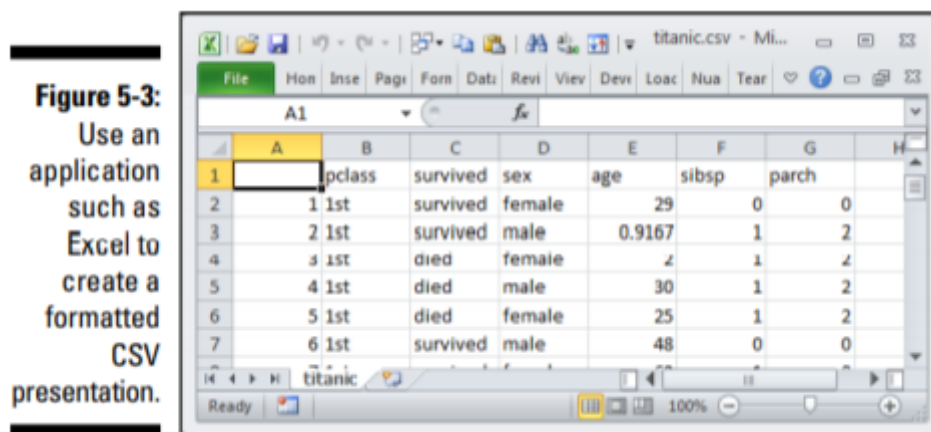
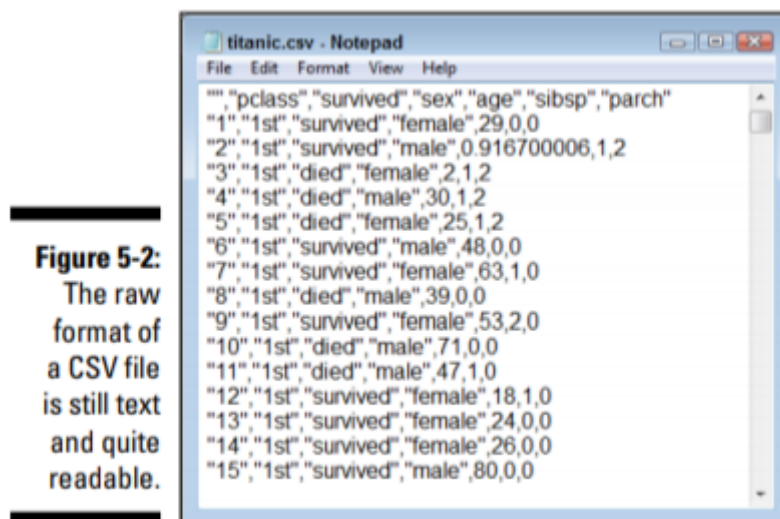
Note:

- Within the pandas library, you find a set of parsers, code used to read individual bits of data and determine the purpose of each bit according to the format of the entire file.
- The code imports the pandas library, uses the `read_table()` method to read `Colors.txt` into a variable named `color_table`, and then displays the resulting memory data onscreen using the `print` function.

Reading CSV delimited format

- A CSV file provides more formatting than a simple text file. In fact, CSV files can become quite complicated. The CSV file used for this example is quite simple:
 - A header defines each of the fields
 - Fields are separated by commas
 - Records are separated by linefeeds
 - Strings are enclosed in double quotes
 - Integers and real numbers appear without double quotes

You can see the raw format using any text editor. Applications such as Excel can import and format CSV files so that they become easier to read.



In [2]:

```
import pandas as pd
titanic = pd.io.parsers.read_csv("Titanic.csv")
x = titanic[['Age']]
print(x)
```

```
      Age
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
..     ...
886   27.0
887   19.0
888    NaN
889   26.0
890   32.0
```

```
[891 rows x 1 columns]
```

To create the output as a list, you simply change the third line of code to read `X=titanic[['age']].values`.

In [3]:

```
x = titanic[['Age']].values
print(x)
```

```
[[22. ]
 [38. ]
 [26. ]
 [35. ]
 [35. ]
 [ nan]
 [54. ]
 [ 2. ]
 [27. ]
 [14. ]
 [ 4. ]
 [58. ]
 [20. ]
 [39. ]
 [14. ]
 [55. ]
 [ 2. ]
 [ nan]
 [31. ]
 ...]
```

Storing content of dataframe to csv file

In [12]:

```
x=pd.read_csv("Titanic.csv")
x1=x[['Name', 'Age']]
print(x1.head())
x1.to_csv('namelist.csv', index=False)
```

	Name	Age
0	Braund, Mr. Owen Harris	22.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0
2	Heikkinen, Miss. Laina	26.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0
4	Allen, Mr. William Henry	35.0

To avoid Python/Pandas creating an index in a saved CSV important way is to set your index value as index=False while you are converting your data frame into CSV

Handling Excel file using pandas

In [13]:

```
x1.to_excel('namelist.xlsx', index=False)
```

In [14]:

```
x1=pd.read_excel('namelist.xlsx')
```

In [15]:

```
x1.head()
```

Out[15]:

	Name	Age
0	Braund, Mr. Owen Harris	22.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0
2	Heikkinen, Miss. Laina	26.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0
4	Allen, Mr. William Henry	35.0

In [19]:

```
x1=pd.read_excel('namelist.xlsx',skiprows=[2,3])  
x1.head()
```

Out[19]:

	Name	Age
0	Braund, Mr. Owen Harris	22.0
1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0
2	Allen, Mr. William Henry	35.0
3	Moran, Mr. James	NaN
4	McCarthy, Mr. Timothy J	54.0

Combine data of multiple datasets

In [84]:

```
x=pd.read_csv("Titanic.csv")  
x1=x[['Name','Age']]  
x2=x1.iloc[:,0]  
x3=x1.iloc[:,1]
```

In [85]:

```
combine=pd.concat([x2,x3],axis=1)
```

In [86]:

```
combine.head()
```

Out[86]:

	Name	Age
0	Braund, Mr. Owen Harris	22.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0
2	Heikkinen, Miss. Laina	26.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0
4	Allen, Mr. William Henry	35.0

In [87]:

```
x=pd.read_csv("Titanic.csv")  
x1=x[['Name','Age']]  
x2=x1.iloc[0:5,:]  
x3=x1.iloc[5:10,:]
```

In [88]:

```
combine=pd.concat([x2,x3],axis=0)
```

In [89]:

combine

Out[89]:

	Name	Age
0	Braund, Mr. Owen Harris	22.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0
2	Heikkinen, Miss. Laina	26.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0
4	Allen, Mr. William Henry	35.0
5	Moran, Mr. James	NaN
6	McCarthy, Mr. Timothy J	54.0
7	Palsson, Master. Gosta Leonard	2.0
8	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	27.0
9	Nasser, Mrs. Nicholas (Adele Achem)	14.0

Concatenating using append

In [20]:

```
x=pd.read_csv("Titanic.csv")
x1=x[['Name', 'Age']]
x2=x1.iloc[0:5,:]
x3=x1.iloc[5:10,:]
```

In [22]:

```
result = x2.append(x3)
print(result)
```

	Name	Age
0	Braund, Mr. Owen Harris	22.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0
2	Heikkinen, Miss. Laina	26.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0
4	Allen, Mr. William Henry	35.0
5	Moran, Mr. James	NaN
6	McCarthy, Mr. Timothy J	54.0
7	Palsson, Master. Gosta Leonard	2.0
8	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	27.0
9	Nasser, Mrs. Nicholas (Adele Achem)	14.0

C:\Users\gcet\AppData\Local\Temp\ipykernel_4356\1916213303.py:1: FutureWarning: The frame.append method is deprecated and will be removed from pandas in a future version. Use pandas.concat instead.

```
result = x2.append(x3)
```

In []: